A Production Bug Caused by N+1 Queries

Dev: 21 queries, 120 ms. **Production:** 201 queries per request and **95% database CPU**.

✕ The code that looked fine

Lazy loading fires one extra query for every row.

```
SELECT * FROM Orders                    ✕ 1×
SELECT * FROM Customers                 ✕ 100×
SELECT COUNT(*) FROM Items              ✕ 100×
```

- Fast with 10 rows in dev
- Slow with 100 rows under load
- No error, no warning

VS

✓ One projected query

Select only what the endpoint returns.

```
1 query                                 // not 201
CPU 95%                                 // now 18%
```

- ✓ No lazy loading
- ✓ No scale-up needed
- ✓ Query count asserted in tests

The fix

</> EF Core

```
var orders = await db.Orders
    .AsNoTracking()
    .Take(100)
    .Select(o ⇒ new OrderDto(o.Id, o.Customer.Name, o.Items.Count))
    .ToListAsync();
```

201 → 1
queries per request

95% → 18%
database CPU

0
servers added

What we learned

- Small test data hides scale bugs, so assert query counts in integration tests.
- Turn lazy loading off. If you need the data, ask for it explicitly.
- Keep LogTo on in dev so every query is visible.
- Before buying hardware, count your queries.



Mohsen Bahrzadeh

.NET Developer | Software Architect

Passionate about Clean Architecture and Scalable Systems

Mohsen.Bahrzadeh@gmail.com

linkedin.com/in/mohsen-bahrzadeh

باگ Production به‌خاطر کوئری‌های N+1

Dev: ۲۱ کوئری و ۲۰۱ کوئری در هر درخواست و **Production:** ۱۲۰ms. CPU دیتابیس ۹۵٪.

کدی که بی‌خطر به نظر می‌رسید

Lazy Loading برای هر ردیف یک کوئری اضافه می‌فرستد.

```
SELECT * FROM Orders                    × 1×
SELECT * FROM Customers                 × 100×
SELECT COUNT(*) FROM Items              × 100×
```

- در Dev با ۱۰ رکورد سریع بود
- زیر بار با ۱۰۰ رکورد کند شد
- نه خطا، نه هشدار

VS

یک کوئری با Projection

فقط چیزی را بخوانید که Endpoint برمی‌گرداند.

```
1 query                                 // not 201
CPU 95%                                 // now 18%
```

- ✓ بدون Lazy Loading
- ✓ بدون نیاز به Scale Up
- ✓ تعداد کوئری در تست Assert می‌شود

راه‌حل

EF Core

```
var orders = await db.Orders
    .AsNoTracking()
    .Take(100)
    .Select(o ⇒ new OrderDto(o.Id, o.Customer.Name, o.Items.Count))
    .ToListAsync();
```

۲۰۱ ← ۱	۹۵٪ ← ۱۸٪	۰
کوئری در هر درخواست	CPU دیتابیس	سرور اضافه

درس‌هایی که گرفتیم

- تست با دادهٔ کم، باگ‌های مقیاس را پنهان می‌کند؛ تعداد کوئری را در Integration Test چک کنید.
- Lazy Loading را خاموش کنید. اگر داده لازم است، صریح بخواهیدش.
- در Dev همیشه LogTo را روشن بگذارید تا هر کوئری دیده شود.
- قبل از خرید سخت‌افزار، اول کوئری‌ها را بشمارید.



Mohsen Bahrzadeh

.NET Developer | Software Architect

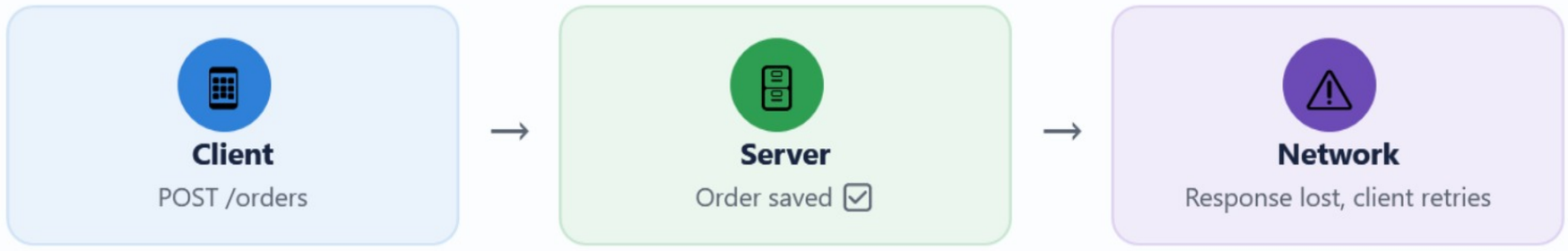
Passionate about Clean Architecture and Scalable Systems

Mohsen.Bahrzadeh@gmail.com

linkedin.com/in/mohsen-bahrzadeh

Retries Without Idempotency Created 3 Orders

A timeout means "I didn't get a response", not "the server did nothing".



Without a key

Every retry is treated as a brand new request.

```
POST /orders              ✗ → order #1
POST /orders (retry)      ✗ → order #2
POST /orders (retry)      ✗ → order #3
```

- One click, three orders
- No error anywhere
- Support finds out first

VS

With an Idempotency-Key

The client reuses one key for every retry of the same intent.

```
Idempotency-Key: 7b1c…        // same key
Order #1                      // returned 3×
```

- ✓ Safe to retry
- ✓ Stored response is replayed
- ✓ Works for payments too

Server side

ASP.NET Core minimal API

```csharp
var existing = await db.IdempotencyRecords.FindAsync(key);
if (existing is not null)
    return Results.Json(existing.Response, statusCode: existing.StatusCode);

db.Orders.Add(order);
db.IdempotencyRecords.Add(new(key, 201, order.ToResponse()));
await db.SaveChangesAsync();   // one transaction, unique index on Key
```

- ✓ Save key + order in one transaction
- ✓ Unique index on the key
- ✓ Same key, different body → 422
- ✓ Give keys a TTL (e.g. 24 h)

> *Whenever you add a retry, ask: what if the previous request actually succeeded?*
>
> — The question to ask



Mohsen Bahrzadeh

.NET Developer | Software Architect

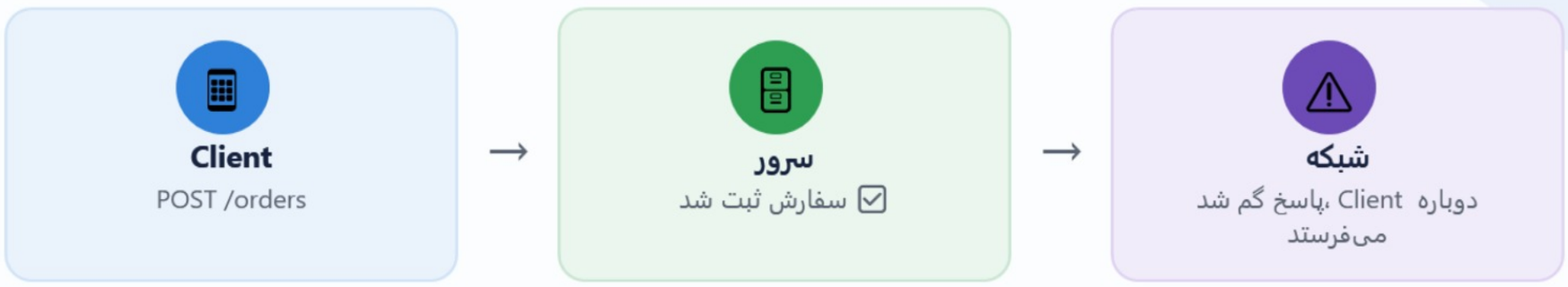
Passionate about Clean Architecture and Scalable Systems

Mohsen.Bahrzadeh@gmail.com

linkedin.com/in/mohsen-bahrzadeh

Retry بدون ۳، Idempotency سفارش تکراری

Timeout یعنی «پاسخ را نگرفتم»، نه «سرور کاری نکرد».



بدون Key

هر Retry یک درخواست کاملاً جدید حساب می‌شود.

```
POST /orders                  ✗ → order #1
POST /orders (retry)          ✗ → order #2
POST /orders (retry)          ✗ → order #3
```

- یک کلیک، سه سفارش
- هیچ خطایی دیده نمی‌شود
- اول از همه پشتیبانی متوجه می‌شود

VS

با Idempotency-Key

Client برای همهٔ Retryهای یک قصد، یک Key می‌فرستد.

```
Idempotency-Key: 7b1c…        // same key
Order #1                      // returned 3×
```

- ✓ Retry امن می‌شود
- ✓ پاسخ ذخیره‌شده دوباره برگردانده می‌شود
- ✓ برای پرداخت هم کار می‌کند

سمت سرور

ASP.NET Core minimal API

```
var existing = await db.IdempotencyRecords.FindAsync(key);
if (existing is not null)
    return Results.Json(existing.Response, statusCode: existing.StatusCode);

db.Orders.Add(order);
db.IdempotencyRecords.Add(new(key, 201, order.ToResponse()));
await db.SaveChangesAsync();    // one transaction, unique index on Key
```

- ✓ ثبت Key و سفارش در یک تراکنش
- ✓ Unique Index روی Key
- ✓ همان Key با بدنهٔ دیگر ← خطای ۴۲۲
- ✓ برای Key یک TTL بگذارید (مثلاً ۲۴ ساعت)

> *هر وقت Retry اضافه می‌کنید بپرسید: اگر درخواست قبلی واقعاً موفق شده بود چه؟*
>
> — سؤالی که باید بپرسید



Mohsen Bahrzadeh

.NET Developer | Software Architect

Passionate about Clean Architecture and Scalable Systems

Mohsen.Bahrzadeh@gmail.com

linkedin.com/in/mohsen-bahrzadeh

The Outbox Pattern: Never Lose an Event

The database and the broker don't share a transaction. SaveChanges succeeded, the message didn't.

✕ Dual write

Two systems, two commits, a crash in between.

```
SaveChangesAsync()        ✕ ☑
bus.Publish(event)        ✕ ✸
```

- Order saved, event lost
- Confirmation email never sent
- Systems silently drift apart

VS

✓ Transactional outbox

Write the event next to the data, publish it afterwards.

```
Orders + OutboxMessages        // one transaction
Worker → Broker                // publishes later
```

- ✓ Atomic write
- ✓ At-least-once delivery
- ✓ Consumers stay idempotent

How it works

API
Saves order + outbox row

→

Worker
Reads and publishes

→

Broker
Consumers handle the event

EF Core

```
db.Orders.Add(order);
db.OutboxMessages.Add(new OutboxMessage(new OrderCreated(order.Id)));
await db.SaveChangesAsync();   // atomic: the order and its event, or neither
```

Atomic
State and event together

Reliable
Nothing lost on a crash

Simple
A table and a worker

Ready-made
MassTransit, CAP, Wolverine



Mohsen Bahrzadeh
.NET Developer | Software Architect
Passionate about Clean Architecture and Scalable Systems

Mohsen.Bahrzadeh@gmail.com
linkedin.com/in/mohsen-bahrzadeh

الگوی Outbox: هیچ رویدادی گم نمی‌شود

دیتابیس و Broker تراکنش مشترک ندارند. SaveChanges موفق شد، ولی پیام نرسید.

Transactional Outbox ✓

رویداد را کنار داده بنویس و بعداً منتشر کن.

```
Orders + OutboxMessages        // one transaction
Worker → Broker                // publishes later
```

- ✓ نوشتن اتمیک
- ✓ تحویل At-least-once
- ✓ Consumerها Idempotent می‌مانند

VS

Dual Write ✗

دو سیستم، دو Commit، و یک کرش بین آن‌ها.

```
SaveChangesAsync()   ✗ ☑
bus.Publish(event)   ✗ ✸
```

- سفارش ثبت شد ولی رویداد گم شد
- ایمیل تأیید هیچ‌وقت نرفت
- دو سیستم بی‌صدا ناسازگار می‌شوند

چطور کار می‌کند

API
ثبت سفارش + ردیف Outbox

→

Worker
خواندن و انتشار

→

Broker
Consumer ها رویداد را پردازش می‌کنند

EF Core

```
db.Orders.Add(order);
db.OutboxMessages.Add(new OutboxMessage(new OrderCreated(order.Id)));
await db.SaveChangesAsync();   // atomic: the order and its event, or neither
```

اتمیک
داده و رویداد با هم

قابل‌اعتماد
با کرش هیچ‌چیز گم نمی‌شود

ساده
یک جدول و یک Worker

آماده
MassTransit، CAP، Wolverine



Mohsen Bahrzadeh
.NET Developer | Software Architect
Passionate about Clean Architecture and Scalable Systems

Mohsen.Bahrzadeh@gmail.com
linkedin.com/in/mohsen-bahrzadeh

Why a Cent Went Missing: double vs decimal

0.1 + 0.2 = 0.30000000000000004. In finance, "almost right" means wrong.

✕ double for money

Binary floating point can't represent most decimal fractions.

```
0.1 + 0.2                    ✕ 0.30000000000000004
0.1 + 0.2 == 0.3             ✕ False
100 / 3 × 3                  ✕ 99.99
```

- Tiny errors accumulate
- Rounding differs per layer
- Remainders silently vanish

VS

✓ decimal + one policy

Exact decimals, explicit rounding, remainders allocated.

```
0.1m + 0.2m                                  // 0.3
Round(2.5m, 0, AwayFromZero)                 // 3
```

- ✓ decimal(18,2) in the database
- ✓ One rounding rule, in one place
- ✓ Parts always sum to the total

Splitting money safely

</> Allocate the remainder

```
public static IReadOnlyList<long> Allocate(long total, int parts)
{
    var share = total / parts;
    var remainder = (int)(total % parts);
    return Enumerable.Range(0, parts)
        .Select(i ⇒ share + (i < remainder ? 1 : 0))
        .ToList();
}
// Allocate(10000, 3) ⇒ [3334, 3333, 3333]   sums to 10000
```

147
units off at month-end

3334 · 3333 · 3333
100.00 split in 3, no cent lost

1
Money type, with currency

Rules for money

- decimal for money, double for measurements.
- Rounding is a business rule: explicit, in one place, documented.
- When splitting, the parts must add up exactly to the total.
- Model Money as its own type, not a plain number.



Mohsen Bahrzadeh
.NET Developer | Software Architect
Passionate about Clean Architecture and Scalable Systems

Mohsen.Bahrzadeh@gmail.com
linkedin.com/in/mohsen-bahrzadeh

چرا یک سنت گم شد: double در برابر decimal

0.30000000000000004. در سیستم مالی «تقریباً درست» یعنی غلط. = 0.2 + 0.1

✓ decimal و یک سیاست مشخص

اعشار دقیق، رُند کردن صریح و تخصیص باقی‌مانده.

```
0.1m + 0.2m                          // 0.3
Round(2.5m, 0, AwayFromZero)         // 3
```

- ✓ در دیتابیس decimal(18,2)
- ✓ یک قاعدهٔ رُند کردن، در یک جا
- ✓ جمع اجزا همیشه برابر کل است

VS

✗ double برای پول

ممیز شناور دودویی بیشتر کسرهای دهدهی را دقیق نشان نمی‌دهد.

`0.1 + 0.2`	✗ `0.30000000000000004`
`0.1 + 0.2 == 0.3`	✗ `False`
`100 / 3 × 3`	✗ `99.99`

- خطاهای ریز انباشته می‌شوند
- رُند کردن در هر لایه فرق دارد
- باقی‌مانده‌ها بی‌صدا گم می‌شوند

تقسیم امن پول

تخصیص باقی‌مانده </>

```
public static IReadOnlyList<long> Allocate(long total, int parts)
{
    var share = total / parts;
    var remainder = (int)(total % parts);
    return Enumerable.Range(0, parts)
        .Select(i ⇒ share + (i < remainder ? 1 : 0))
        .ToList();
}
// Allocate(10000, 3) ⇒ [3334, 3333, 3333]   sums to 10000
```

1

نوع Money، همراه با ارز

3333 · 3333 · 3334

تقسیم ۱۰۰.۰۰ بین ۳، بدون گم شدن سنت

147

واحد اختلاف در پایان ماه

قواعد کار با پول

- decimal برای پول، double برای اندازه‌گیری.
- رُند کردن یک قاعدهٔ کسب‌وکار است: صریح، یکجا و مستند.
- در تقسیم پول، جمع اجزا باید دقیقاً برابر کل باشد.
- Money را به‌عنوان یک نوع جداگانه مدل کنید، نه یک عدد ساده.



Mohsen Bahrzadeh

.NET Developer | Software Architect

Passionate about Clean Architecture and Scalable Systems

Mohsen.Bahrzadeh@gmail.com

linkedin.com/in/mohsen-bahrzadeh